TERCER LABORATORIO

Etapa 6: Preprocesado (en el fichero preprocesado.py)

Objetivo: Definir clases para realizar preprocesado de los datos. Dependiendo del modelo de ML que se vaya a usar puede ser adecuado un determinado preprocesamiento de los datos. Esta etapa incluye tareas como qué hacer con datos que falten, quitar "outliers", normalización, seleccionar ejemplos o atributos adecuados, etc.

1. Clase Preprocesamiento (abstracta)
   - Implementa sin código los métodos **ajustar**, **transformar_dataSet** y **transformar_Registro**.

2. Clase NormalizadorMaxMin
   - Tiene dos atributos que son listas de máximos y mínimos inicializadas a vacío.
   - Un método **ajustar** que recibe un DataSet y calcula las lista con los máximos y mínimos mediante la función correspondiente de DataSet.
   - Un método **transformar_registro** que recibe un Registro y devuelve otro normalizado.
   - Un método **transformar_dataSet** que recibe un DataSet y devuelve otro con sus registros normalizados usando la correspondiente función de Registro.
   -

3. Clase NormalizadorZ_Score
   - Esta transformación conocida como estandarización modifica los valores de los atributos para que se distribuyan según una distribución normal de media 0 y desviación típica 1 (busca información sobre esta transformación).
   - Debes modificar las clases Registro y DataSet para añadir los métodos: **estandarizar** para Registro y para DataSet **calcular_medias_desviaciones**.
   - La clase tiene dos atributos que son listas de medias y desviaciones inicializadas a vacío.
   - Un método **ajustar** que recibe un DataSet y calcula las medias y desviaciones mediante el correspondiente método de DataSet.
   - Un método **transformar_registro** que recibe un Registro y devuelve otro estandarizado.

- Un método **transformar_dataSet** que recibe un DataSet y devuelve otro con sus registros estandarizados usando el correspondiente método de Registro.

Etapa 7: Validación (en el fichero validación.py)

Objetivo: Definir clases para realizar el aprendizaje para un modelo de clasificación o de regresión y valorar cuánto de bueno (o malo) es. Para validar un modelo debe separarse la fase de entrenamiento (aprendizaje) de la fase de prueba o test (valoración). Lo habitual es dividir el conjunto de datos disponible en dos partes, llamadas habitualmente ***train*** y ***test***, de forma que el modelo aprende de unos datos y se valora por otros distintos. Con esto se evita que los modelos se sobreajusten a los datos de train y no generalice correctamente cuando se "enfrente" a datos que no ha "visto" previamente.

4. Clase Validación (abstracta)
   - Esta clase implementará distintas formas de validación como "hold-out", "cross-validation" o "leaving-one-out" y tendrá un método abstracto **calcular_metricas,** que recibe una lista de etiquetas (o valores) predichos por el modelo y reales (la etiqueta o valor presente en el DataSet) y calcula una métrica que nos diga si las predicciones son buenas o no. Como las métricas posibles dependen de si estamos ante un problema de clasificación o de regresión, este método se redefinirá en unas clases hijas ValidacionClasificacion y ValidacionRegresion
   - Defina el método **validacion_simple** (busque información sobre validación hold-out) que recibe un objeto Modelo, un objeto DataSet, un porcentaje de división entre train y test y un objeto de tipo normalización (por defecto None). Este método debe dividir el DataSet en dos partes según el porcentaje dado como parámetro (use el método **crear_subconjunto** de DataSet). Si existe normalizador debe modificar tanto los datos de train como de prueba de acuerdo a la transformación. Después debe invocar al método **entrenar** del Modelo pasado como argumento y usando el método **predecir** debe anotar las predicciones que el modelo hace sobre todos los registros de test, Finalmente debe invocar al método **calcular_metricas** que recibe los valores predichos y los valores reales para calcular la métrica del rendimiento.

- Por último, la clase implementa el método **validación_cruzada** (busque información sobre cross_validation) que recibe un Modelo, un DataSet, un número de bolsas (m) y test y un objeto de tipo normalización (por defecto None). Este método dividirá el DataSet en las m bolsas indicadas y repetirá el proceso m veces entrenando con m-1 bolsas y testando con una. El resultado de la validación deberá ser la media de las m ejecuciones.

5. Clases ValidacionClasificacion y ValidacionRegresion
   - Estas clases deben concretar el método **calcular_metricas**, de forma que para clasificación calcule la accuracy (aciertos/errores) y para regresión calcule el MAE (media de los errores absolutos).

Etapa 8: Nuevos modelos (en modelos.py)

6. Clase Regresor_kNN (hereda de Modelo)
   - El constructor recibe como argumentos el valor de k y la distancia
   - El método **predecir** debe invocar al método k_vecinos de Registro usando datos_entrenamiento. Este método devolverá la media del atributo valor de los vecinos del Registro que se da como argumento.
   -
7. Clase Clasificador_centroide (hereda de Modelo)
   - Este clasificador calcula los centroides de los Registros de cada etiqueta del DataSet y asigna a un Registro de test, la etiqueta correspondiente al centroide más cercano.

8. Clase Regresor_lineal_multiple (hereda de Modelo)
   - Este regresor calcula mediante una aproximación heurística una recta de regresión múltiple. Como la recta ya incorpora pesos en los coeficientes, no necesita normalización previa para su ejecución.

Las dos últimas etapas son voluntarias.

Etapa 9: Nuevas métricas (en validacion.py)

9. Se ha proporcionado una solo métrica para clasificación (accuracy) y otra para regresión (MAE). Investigue qué otras métricas hay e impleméntelas de forma adecuada a la estructura que tenemos.

Etapa 10: Nuevos preprocesados (en validacion.py)

10. Añada a DataSet un método que reciba un conjunto de índices y devuelva otro DataSet en el que los atributos dados por esos índices no estén. Compruebe que los índices están en el rango lógico.

11. Añada a validación.py una clase SeleccionAtributos con un método selección_correlacion que reciba un porcentaje p y devuelva los índices de los atributos con mayor correlación (coeficiente de Pearson) con el valor objetivo. Debe devolver un porcentaje p de los atributos más correlados.

12. Investigue métodos para selección de ejemplos y de selección de atributos en clasificación. Añada alguno(s) como clases a preprocesado.py. Si nos da tiempo volveremos sobre esto al final del cuatrimestre.

13. Igualmente, la distancia ponderada añade un problema de optimización habitual en el ML. ¿Qué valores deben tener los pesos de la distancia ponderada para mejorar los modelos? También es posible volvamos sobre este tema más adelante.